PUBLISHED Jul 27, 2025
ARTICLE UWTN 2025-003
VERSION 1.0.0
LICENCE CC-BY-4.0
ARCHIVED 2026-08-10T04:19:43Z
SOURCE https://www.underworldcode.org/how-many-processors-should-we-use-to-solve-problem-x/

How many processors should we use to solve Problem X?

Juan Carlos Graciosa[1], and **Louis Moresi**[1]

[1]Australian National University

DOI https://doi.org/10.6084/m9.figshare.33193554

Abstract

Parallel computation puts many CPUs to work on solving a problem much more quickly than one CPU alone. But this only works if the tasks are carefully scheduled and the additional CPUs are not waiting around for something to do. How do we choose the right number of processors for a given problem ?

Keywords Documentation, Tricks of the Trade, Underworld Code

Running geodynamic models on a single CPU/processor (i.e. serial) is time-consuming and limits us to low resolution. Underworld is build from the ground-up as a parallel computing solution which means we can easily run large models on high performance computing clusters (HPC); that is, sub-divide the problem into many smaller chunks and use multiple processors to solve each one, taking care to combine and synchronise the answers from each processor to obtain the correct solution to the original problem.

Parallel computation can reduce time we need to wait for the our results to be computed but it does happen at the expense of some overhead The overhead does depend on the nature of the computer we are using but typically we need to think about:

- Code complexity: any time we manage computations across different processors, we have additional coding to reassemble the calculations *correctly* and we need to think about many special cases. For example, integrating a quantity of the surface of a mesh: many processes contribute, some do not, the results have to be computed independently then combined.
- Additional memory is often required: to manage copies of information that lives on / near boundaries, to store the topology of the decomposed domain and to help navigate the passing of information between processes.
- The time taken to synchronise results and the work required to keep track of who is doing what, when they are done, and in making sure everyone waits for everyone else. There is a time-cost in actually sending information as part of a synchronisation and a computational cost in ensuring that work is distributed efficiently.

You will want to know in advance whether there is any speed benefit to running in parallel, and, if you are being billed on a cycle-by-cycle rate, you want to be able to estimate the time saved balanced against the additional computational cost.

Enter the strong scaling test! In doing strong scaling tests, the size of the problem is kept constant, while the number of processors is increased. The reduction in run-time due to the addition of more processors is commonly expressed in terms of the *speed-up*:

$$\text{speedup} = \frac{t(N_{ref})}{t(N)} \tag{1}$$

where $t(N_{ref})$ is the run-time for a reference number of processors, N_{ref}, and $t(N)$ is the run-time when N processors are used. In the ideal case, N additional processors should contribute all of its resources in solving the problem and *reduce* the compute time by a factor of N relative to the reference run time. For example, using $2N_{ref}$ processors will ideally halve the run-time resulting to a *speed-up = 2.*

1. Underworld3 test problems

We employ 3D Poisson and Stokes problems to demonstrate strong scaling with Underworld3. The non-linear 3D Cartesian Poisson problem follows:

$$\nabla.((1+u^2(x,y,z))\nabla u(x,y,z)) = f(x,y,z) \tag{2}$$

with analytical solution $u(x,y,z) = \sin(\pi x)\sin(\pi y)\sin(\pi z)$ and Dirichlet ($u=0$) boundary conditions. On the other hand, the 3D spherical Stokes problem follows:

$$-\nabla.\Big[\frac{\eta}{2}(\nabla \boldsymbol{u}(r,\theta,\phi) + \nabla \boldsymbol{u^T}(r,\theta,\phi)) - p(r,\theta,\phi)\Big] = -g\rho'(r,\theta,\phi)\hat{\boldsymbol{r}} \tag{3}$$

$$[\nabla.\boldsymbol{u}(r,\theta,\phi)] = 0 \tag{4}$$

with no slip boundary conditions and a smooth density distribution. For more information on the Stokes problem, see **case X** of Kramer et al., (2021). Solutions of the Poisson and Stokes problems are presented in Figures 1a and 1b, respectively.

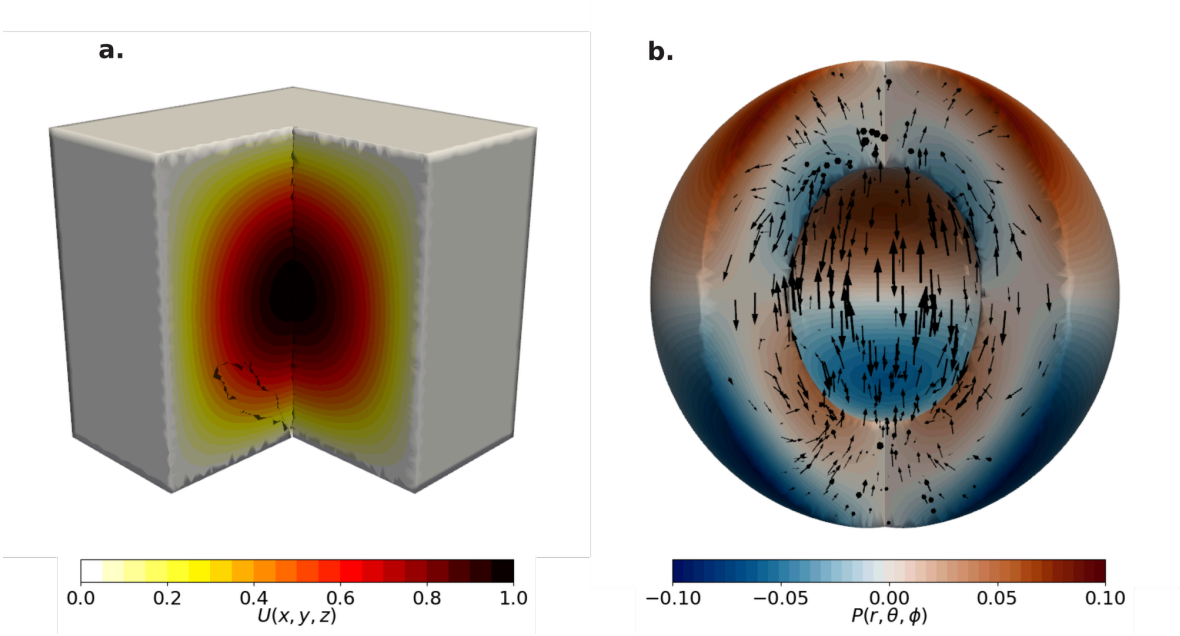
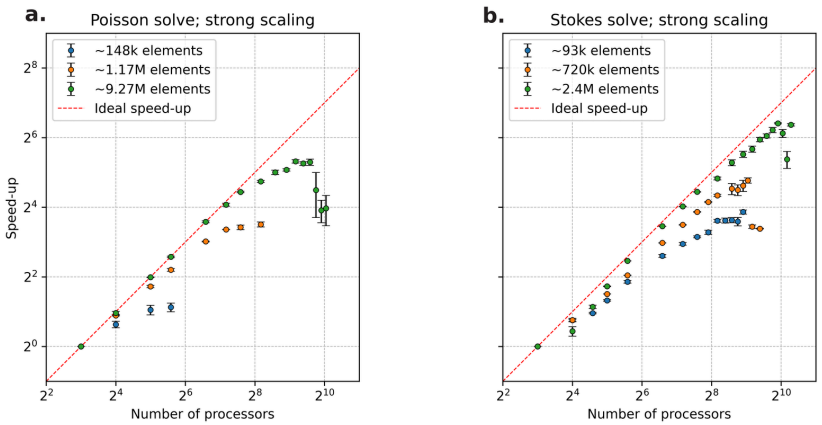


Figure 1: Figure 1. Solutions of the a.) Poisson and b.) Stokes models.

2. Results and Insights

HPC Systems | NCI —

Shown in Figures 2a and 2b are the speed-up obtained in NCI *Gadi* versus the number of processors used for the Poisson and Stokes solves, respectively. Three different resolutions were tested for the Poisson (∼148k, ∼1.17M, and ∼9.27M elements) and Stokes (∼93k, ∼720k, ∼2.4M) problems. Both problems have $N_{ref} = 8$. Additionally, the ideal speed-up with unit slope is given by the red dashed lines. In all cases, the observed speed-up deviates slightly from the ideal, likely due to the fraction of the procedure that is serial as discussed in Amdahl's law. This is particularly evident for smaller problems (i.e., ∼148k elements for Poisson and ∼93k elements for Stokes). Eventually, using more processors (and compute nodes) does not increase the speed-up and can even result to worse performance. At this point, the communication overhead resulting from the data traffic between more compute nodes is significant enough. We also note that although the Poisson problems generally have more elements, they have fewer degrees of freedom so the speed-up plateaus earlier relative to the Stokes problems.



So how can we use strong scaling to determine the optimal number of processors to solve a problem? If turnaround time is prioritized, then one should aim for the highest speed-up possible. If one intends to do more runs of the model used in the strong scaling test, then the number of processors corresponding to the largest speed-up prior to degradation should be a good choice. If not, then one can calculate the elements per processor corresponding to the largest speed-up and use this to estimate the optimal number of processors for the same model but with different resolution. As an example, running the spherical Stokes model with $\sim$ 2.4M elements will have optimal speed-up when 1000 processors are used (Figure 2b). This corresponds to $\sim$ 2.4k elements per processor. With this estimate, optimally running a similar spherical Stokes model but with $\sim$720k elements (i.e. lower resolution), should need $\sim$300 cores. Lastly, if turnaround speed is not a priority, then one should aim to use the elements per processor that are within the linear speed-up regime to ensure that the computing resources are not wasted.

These results are a guide to potential performance available when running Underworld codes in parallel. Speed up / performance graphs are averages over many runs, and the exact details depend on the architecture, the communications hardware, queueing policies and other details that you might need to ask your HPC team to explain. If you are seeing wildly unexpected results, get in touch by leaving us a message in the comments or on the GitHub issue trackers.